

Tópicos em IA - Turma 2

AI4Good

Prática - MLP

noções complementares *de deep learning*

inicialização

inicialização dos pesos da rede

a priori os pesos da rede são inicializados aleatoriamente

alternativamente a inicialização Xavier ou Glorot

é uma técnica de inicialização de pesos para aprimorar o treinamento

a proposta é manter um
equilíbrio na variância das ativações
entre as camadas

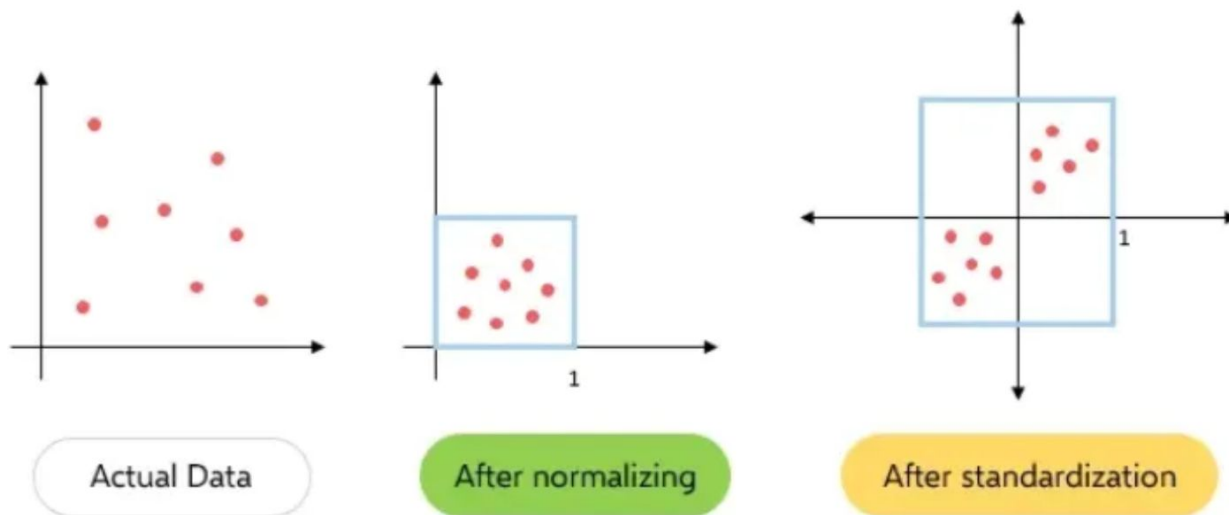
é particularmente eficaz na
prevenção de gradientes que desaparecem ou explodem,
problemas comuns em redes profundas.

[tf.keras.initializers.GlorotUniform](https://keras.io/initializers/#glorot-uniform) | TensorFlow v2.16.1

pré-processamento dos dados

normalização e padronização

pré-processamento dos dados pode ajudar



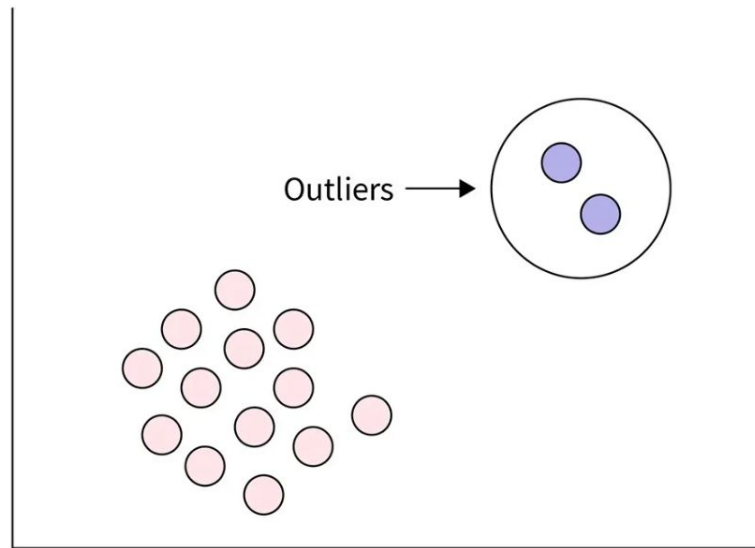
[Is Data Normalization Always Necessary Before Training Machine Learning Models? A Deep Dive into Use Cases | by Vivek Murali | Medium](#)

outliers

outliers

outliers podem prejudicar bastante os dados se forem aplicadas normalizações

escolha bem ou experimente diferentes formas de pré-processamento para não sofrer com esse problema

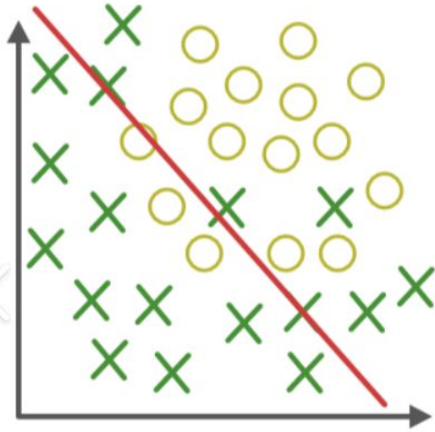


[Outliers: O que são Pontos Fora da Curva e Como Tratá-los?](#)

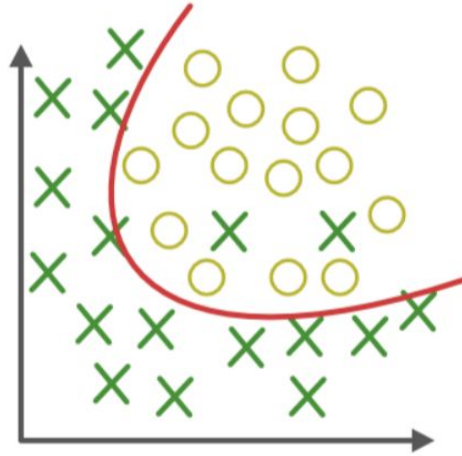
fitting

redes pouco ou muito 'poderosas'

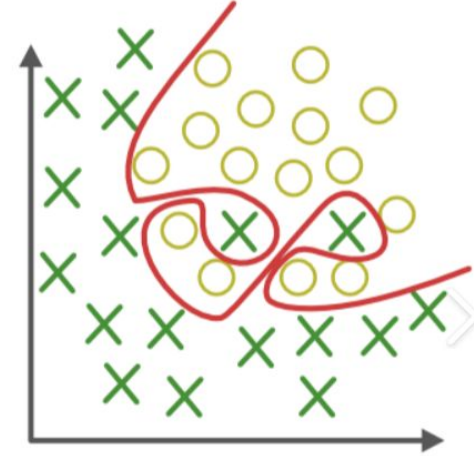
fitting



Under-fitting
(too simple to
explain the variance)



Appropriate-fitting



Over-fitting
(forcefitting--too
good to be true) $\partial\partial$

épocas (*epoch*)
lotes (*batch*)
iterações (*iteration*)

epoch, batch, iteration

lote (**batch**): um subconjunto dos dados de treinamento processados juntos antes do otimizador atualizar os pesos.

iteração (**iteration**): uma etapa de atualização de parâmetro; ou seja, cada vez que a rede conclui uma passagem para frente e para trás em um lote

época (**epoch**): uma passagem completa por cada amostra de treinamento, geralmente composta de muitas iterações

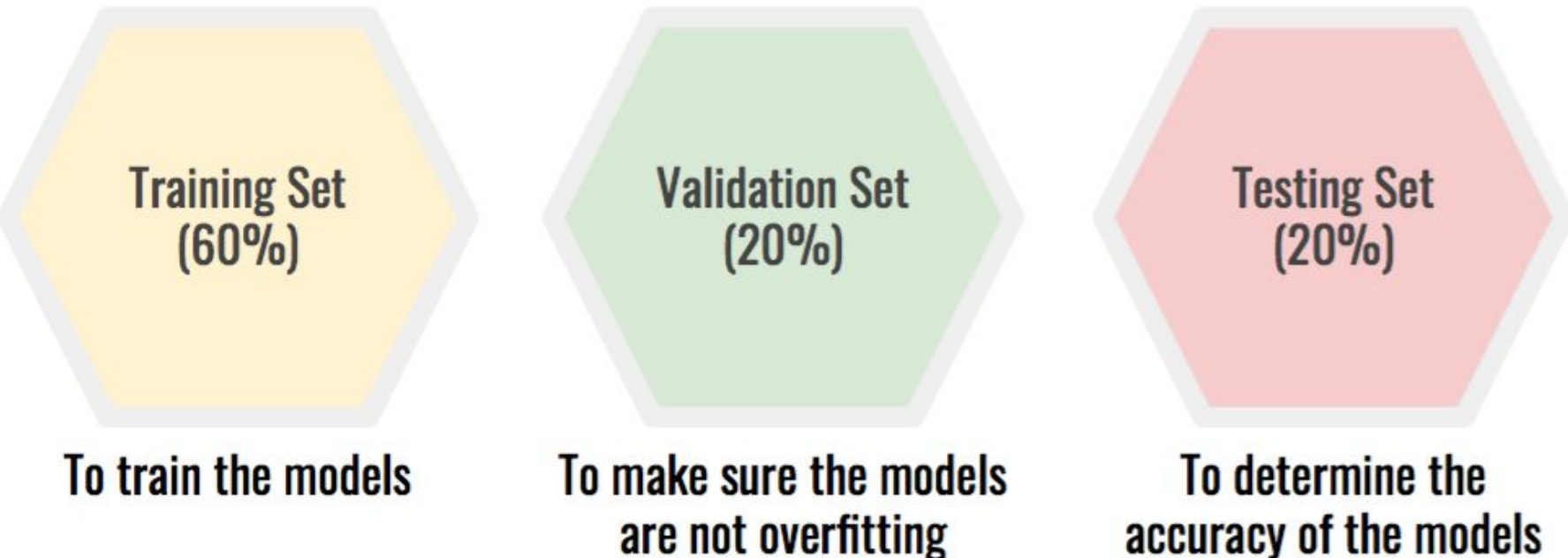
hiperparâmetros

hiperparâmetros

- **taxa de aprendizagem (*learning rate*):** 0.03, 0.01, 0.003, 0.001
- **arquitetura:** camadas escondidas e neurônios em cada uma
- **função de ativação:** ReLU, tanh, etc
- **número de épocas:** 10, 20, 50, 100
- **normalização, padronização**
- **dropout**
- **regularização**

divisão dos dados
treinamento, validação e teste

conjuntos de treino, validação e teste



**Training Set
(60%)**

To train the models

**Validation Set
(20%)**

**To make sure the models
are not overfitting**

**Testing Set
(20%)**

**To determine the
accuracy of the models**

[What is the difference between Training dataset, Testing Dataset, Validation dataset? What is the Common Ratio? | i2tutorials](#)

conjuntos de treino, validação e teste

SKIPPED — desenvolvedores geralmente separar os dados dos datasets em porções para não enviesar o modelo, justamente evitar overfitting, deixá-lo mais generalizável

treino: usado para de fato treinar o modelo

validação: usado para refinar os hiperparâmetros, taxa de aprendizagem, número de épocas, etc

teste: usado de fato para reportar a acurácia do modelo

[What is the difference between Training dataset, Testing Dataset, Validation dataset? What is the Common Ratio? | i2tutorials](#)

conjuntos de treino, validação e teste

desenvolvedores geralmente separam os dados dos datasets em porções para não enviesar o modelo, justamente evitar overfitting, deixá-lo mais generalizável

treino: usado para de fato treinar o modelo

validação: usado para refinar os hiperparâmetros, taxa de aprendizagem, número de épocas, etc

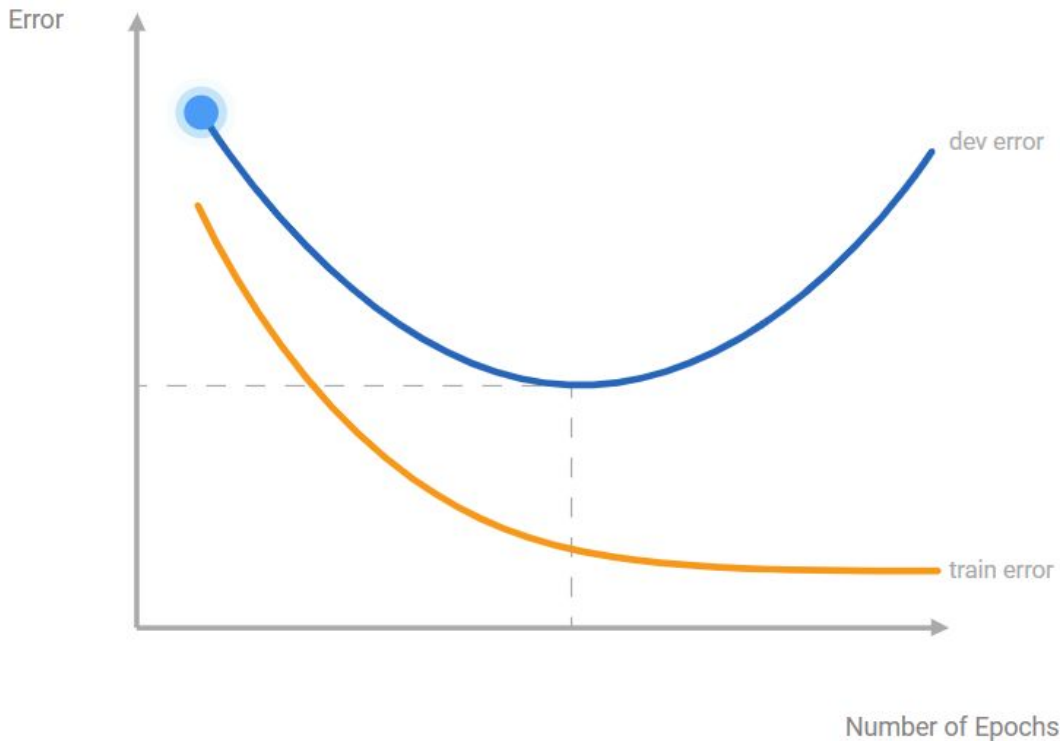
teste: usado de fato para reportar a acurácia do modelo

[What is the difference between Training dataset, Testing Dataset, Validation dataset? What is the Common Ratio? | i2tutorials](#)

regularização
early stopping

early stopping

para evitar o overfitting o treinamento é parado quando o erro no conjunto de validação começa a aumentar



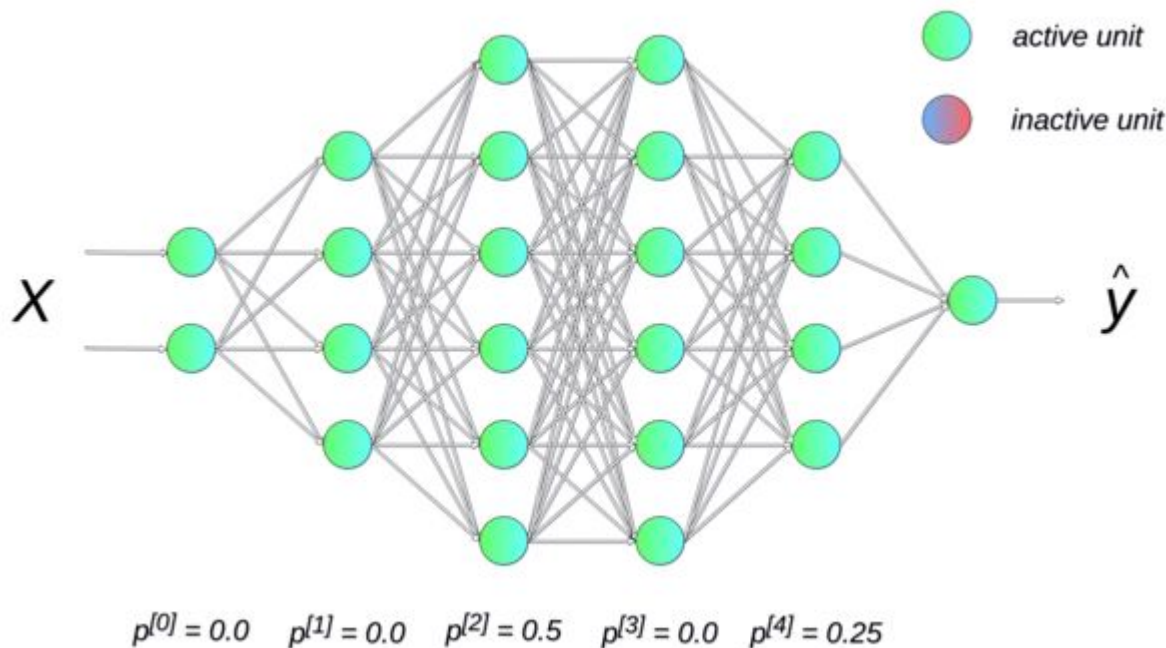
regularização *dropout*

dropout

SKIPPED — uma **medida de regularização** na qual evita-se overfitting

dropout é um algoritmo que especifica que a cada iteração de época do treino os neurônios possuem uma probabilidade de serem removidos (não utilizados) para a aprendizagem

geralmente a probabilidade ideal fica em torno de 20%



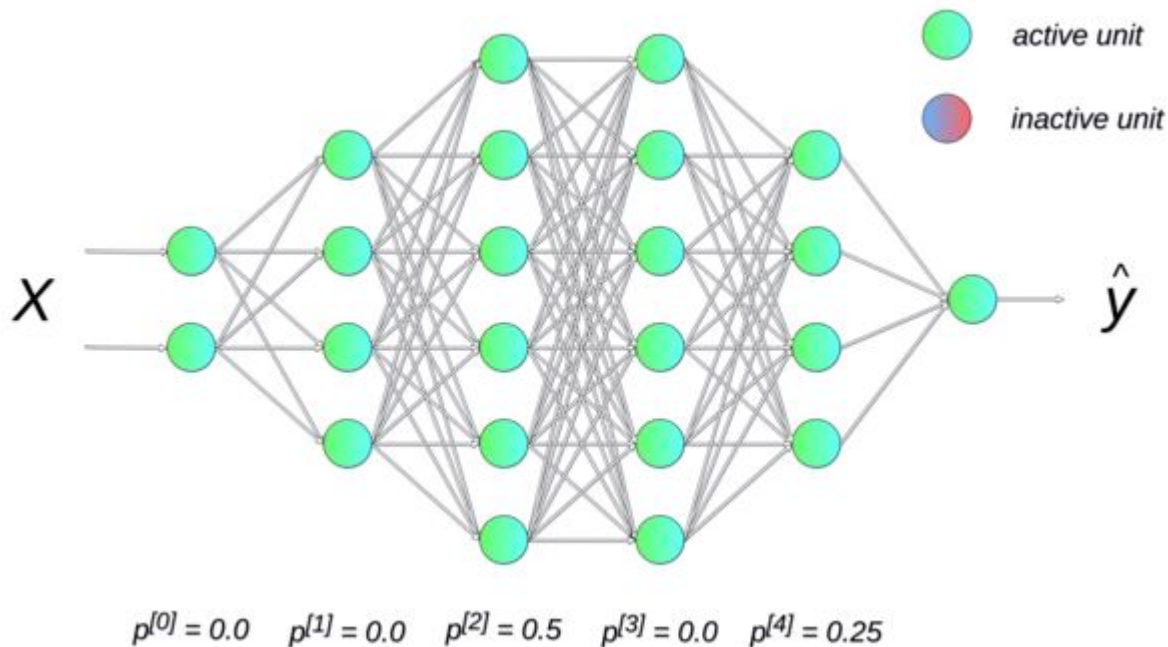
dropout

uma **medida de regularização** na qual evita-se overfitting

dropout é um algoritmo que especifica que a cada iteração de época do treino os neurônios possuem uma probabilidade de serem removidos (não utilizados) para a aprendizagem

geralmente a probabilidade utilizada fica entre 20% e 50%, dependendo da rede e da camada

[Aula 18 b Redes Neurais com PyTorch.ipynb - Colab](#)

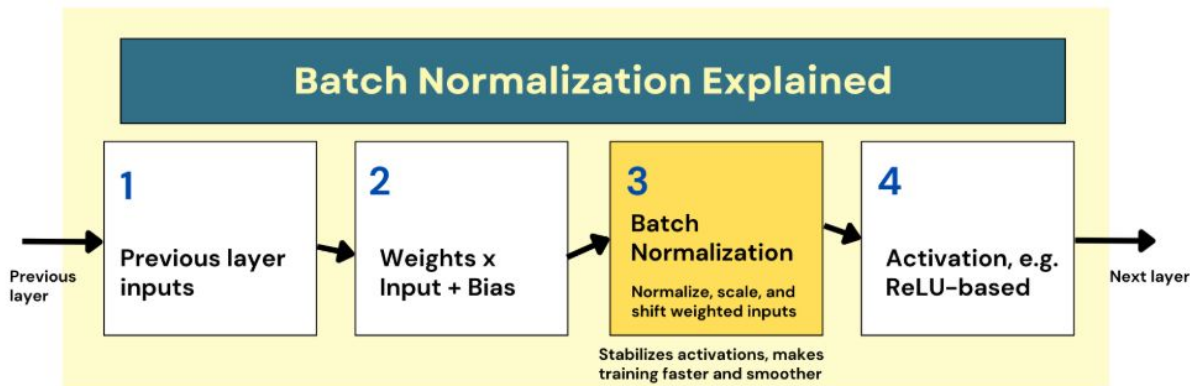


regularização

batch normalization

batch normalization

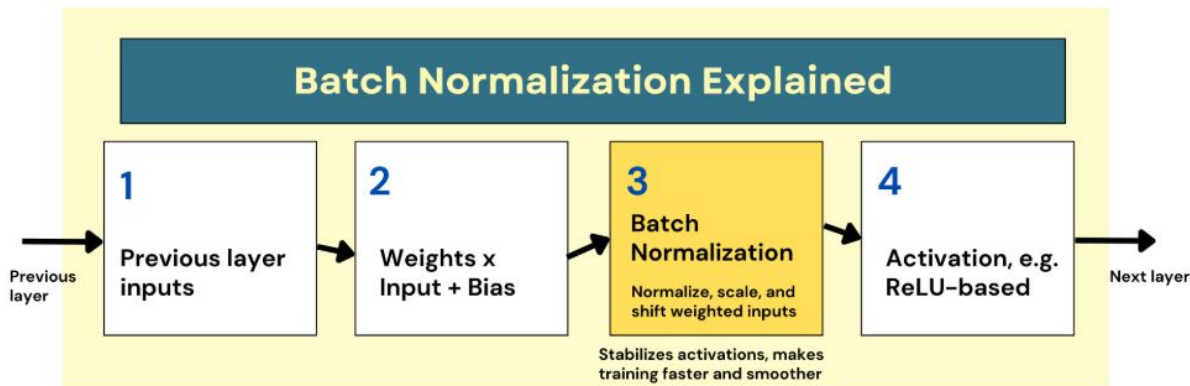
SKIPPED — outra
**medida de
regularização** na
qual se busca
evitar grandes
discrepâncias nos
valores calculados
durante o
treinamento



[A Gentle Introduction to Batch Normalization - MachineLearningMastery.com](https://machinelearningmastery.com/a-gentle-introduction-to-batch-normalization/)

batch normalization

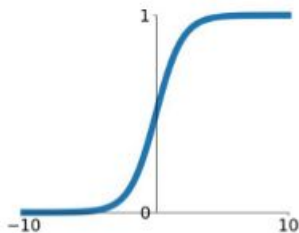
normaliza **as**
entradas de cada
camada,
estabilizando e
acelerando o
treinamento; de
quebra, age como
uma regularização



funções de ativação
vanishing gradients
exploding gradients
dying ReLU

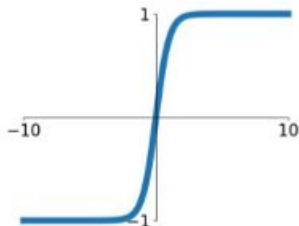
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



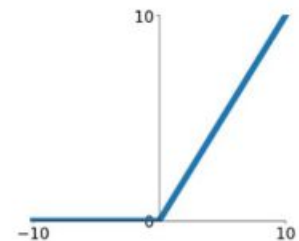
tanh

$$\tanh(x)$$



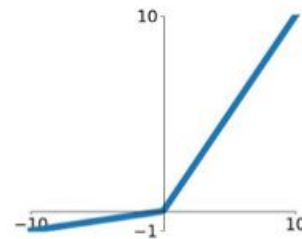
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

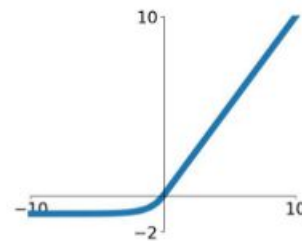


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

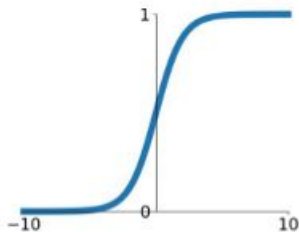
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



vanishing gradients

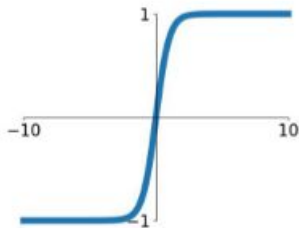
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



tanh

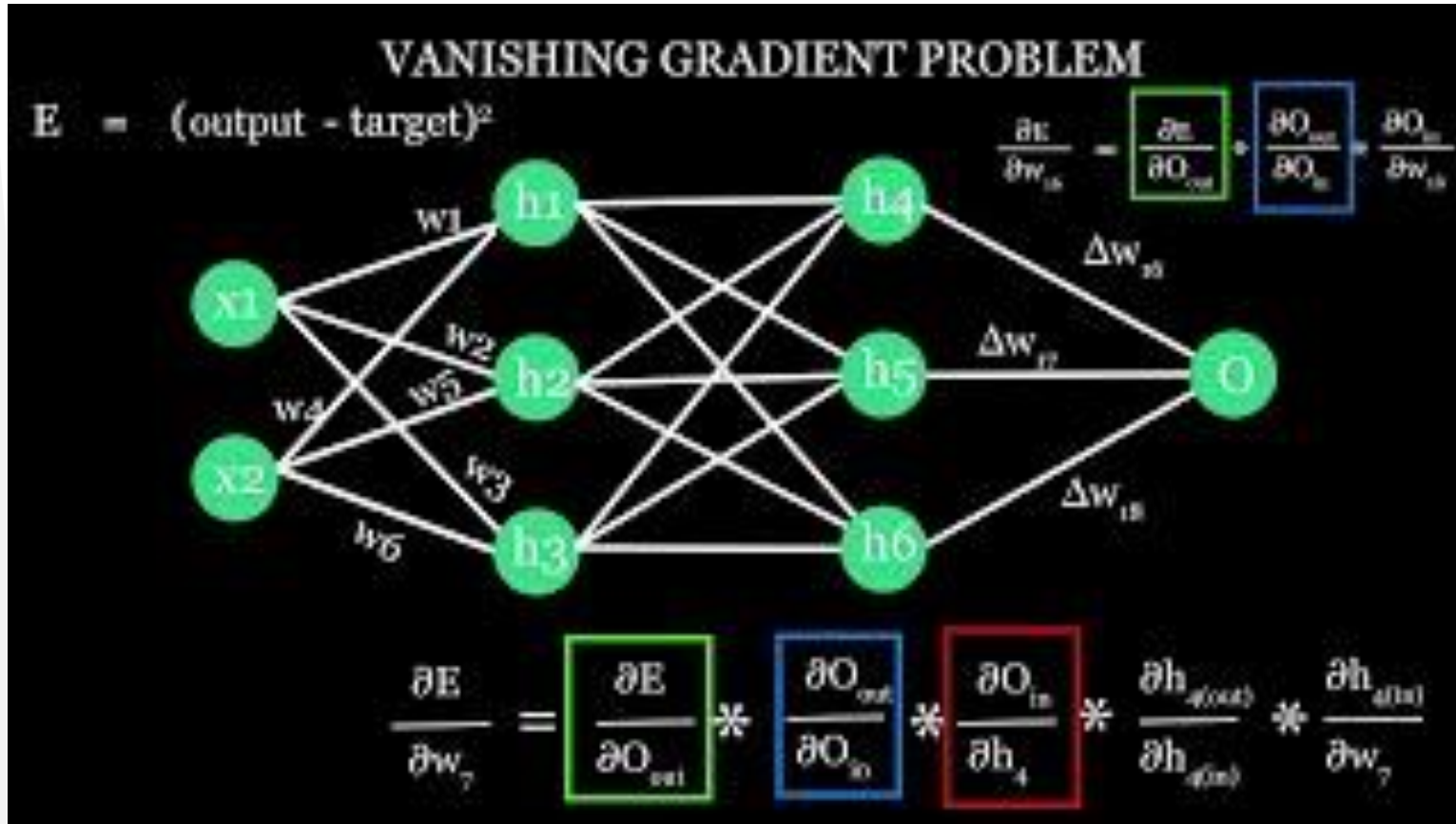
$$\tanh(x)$$



as derivadas (gradientes) das funções Sigmoid e tanh variam entre 0 e 0.25 e 0 e 1 respectivamente

durante o processo de *backpropagation* os gradientes podem se tornar progressivamente menores ao se encaminhar para as primeiras camadas

vanishing gradients



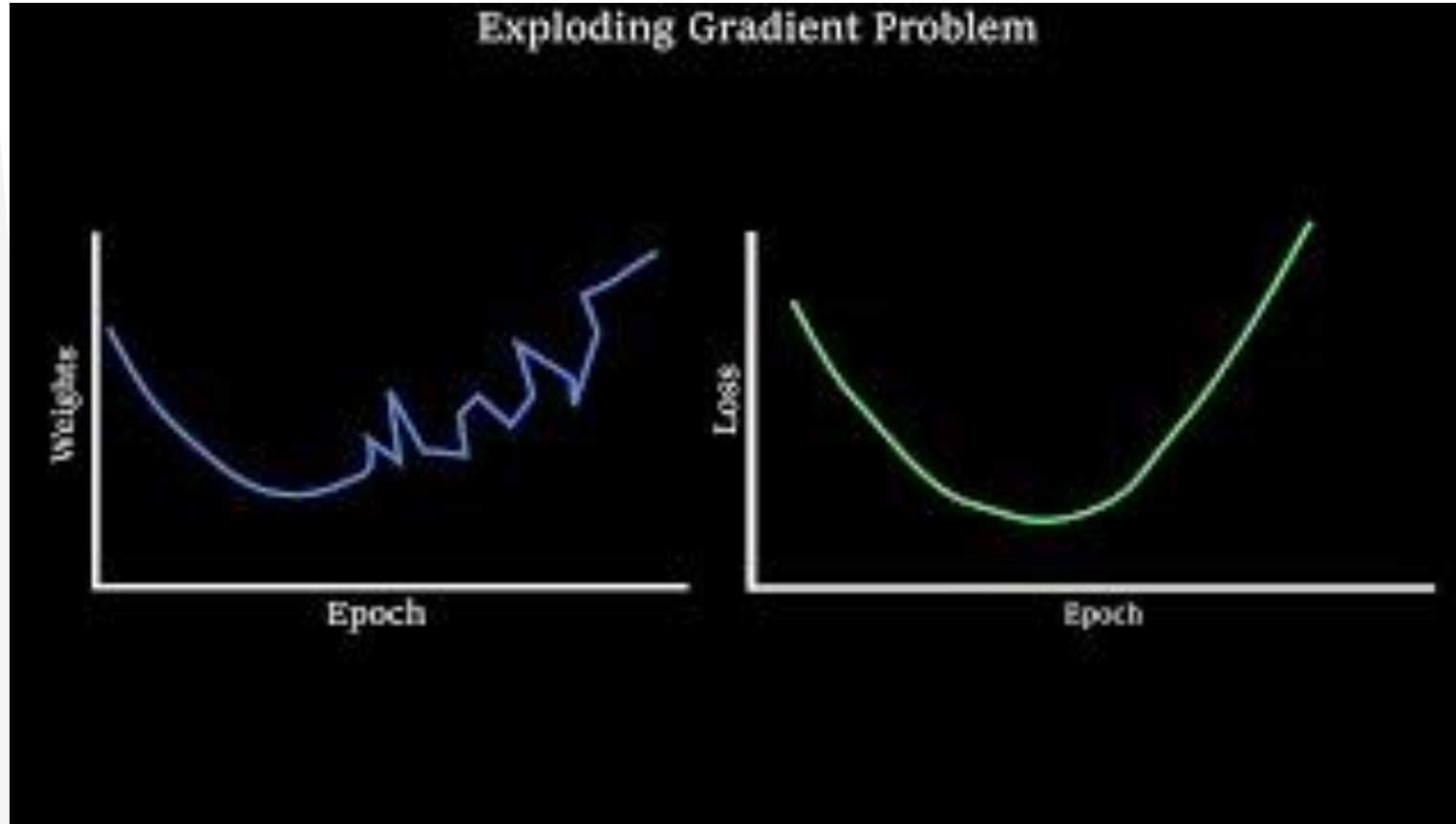
exploding gradients

por outro lado, outras funções de ativação que permitem que gradientes durante o processo de *backpropagation* retornem valores maiores do que 1 potencialmente geram um crescimento exponencial dos gradientes.

soluções são:

- *gradient clipping*, 'cortar' os valores dos gradientes de acordo com limiares estabelecidos
- *batch normalization*, normaliza as entradas de cada camada. Isso pode ajudar a estabilizar e acelerar o processo de treinamento

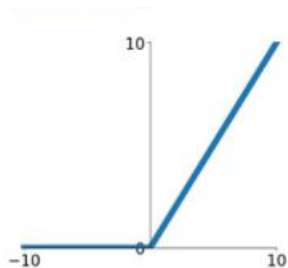
exploding gradients



dying ReLU

ReLU

$$\max(0, x)$$

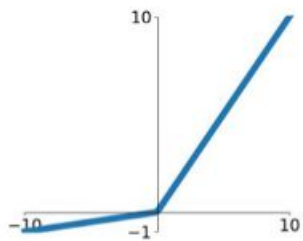


por cortar os valores negativos o uso da função **ReLU** pode gerar o problema do “dying ReLU”

o problema ocorre quando alguns neurônios podem ficar inativos durante o treinamento

Leaky ReLU

$$\max(0.1x, x)$$



a função **Leaky ReLU**, uma variação do ReLU, permite um pequeno gradiente quando a entrada é negativa

prática

objetivo

codificar uma MLP do zero (*from scratch*) e
visualizar o processo de
feed forward e *back propagation*

especificações

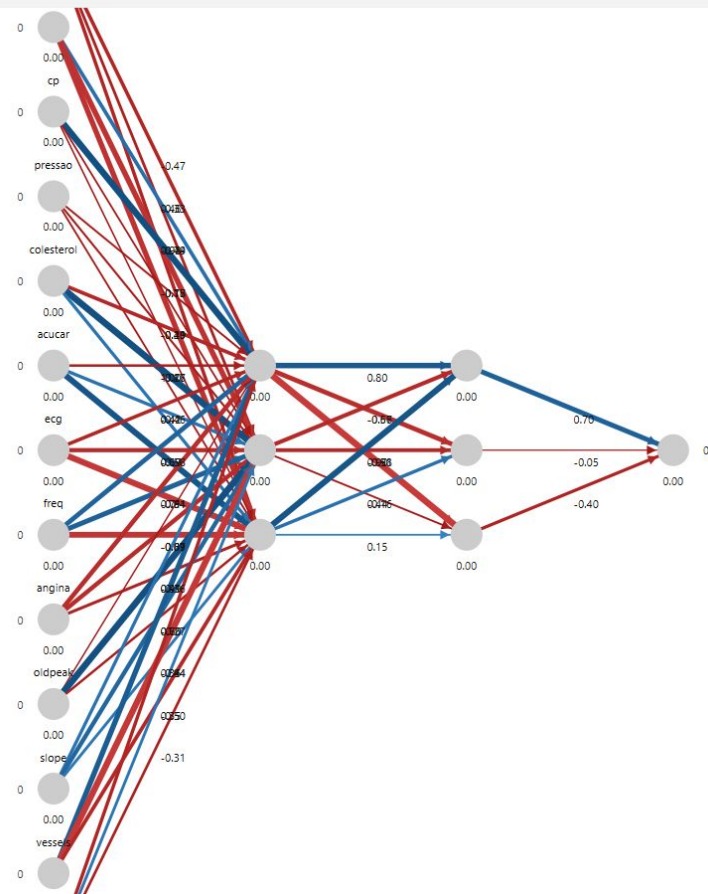
1. codificar uma **rede neural** de múltiplas camadas (MLP - Multi-layer Perceptron) a partir da estrutura de um grafo
2. implementar a função de ***feed forward*** e ***back propagation***
3. utilizar a **base de dados** "[Heart Disease](#)" para treinar (80% pra treino, 20% teste) a rede e em seguida realizar a inferência
4. **demonstrar/visualizar** o progresso de treinamento e da inferência usando alguma API para mostrar o grafo e os pesos sendo atualizados em tempo real

desafio (+0.3 na VA1):

1. leaderboard no dataset Diabetes 130-US Hospitals for Years 1999-2008 (80% pra treino, 20% teste)

<https://archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008>

exemplo

**Camadas**

Tamanho 3 Posição 1

Adicionar Camada

Índice 1

Remover Camada

Neurônios

Camada 1

Adicionar Neurônio

Camada 1 Índice 0

Remover Neurônio

Entrada

Ex: 0.1, -0.2, 1.0, ... Aplicar

Ativação identity

Dataset

Carregar heart.csv

Próximo sample

Fast Forward (todas)

Execução

▶

Passo

||

Reset

Camadas: [13, 3, 3, 1]

Treino

LR 0.0100

Train Step

Train Época

Ver Loss

entrega

mostrar o seu **harness** funcionando

mostrar a sua **estrutura de pastas**,
onde fica o seu projeto e seus arquivos .md

mostrar o exercício versionado no
git e usando **gitflow**

cada equipe terá que mostrar
o **código rodando**
o **treinamento** e
a **inferência**

adicionar um **slide abaixo do último**,
copiando os mesmos campos e
preenchendo eles

ranking

1. Pontuação : Nomes

resultados

0.885 : Lucas Silva Figueiredo

accuracy: 0.885

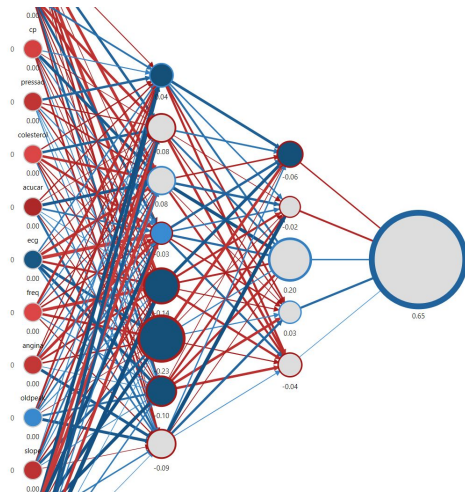
learning rate: 0.01

activation: relu

architecture: 13 | 8 | 5 | 1

pre-processing: standardize

comentários: o uso da 'padronização' (standardize) ajudou a resolver o problema de gradientes explodindo no meu caso



Camadas

Tamanho 2 Posição 1

Adicionar Camada

Índice 1

Remover Camada

Neurônios

Camada 1

Adicionar Neurônio

Camada 1 Índice 0

Remover Neurônio

Entrada

000, 168.000, 0.000, 1.000, 2.000, 2.000, 3.000 Apicar

Ativação relu

Dataset

Carregar heart.csv Próximo sample

Fast Forward (todas)

Pré-processamento standardize

Salvar JSON Carregar JSON

Execução

Passo II Reset

Época: loss_med=0.04729 acc=0.885

Treino

LR 0.0100

Train Step Train Época

Ver Loss

0.833 : Davi Pires Aquino

accuracy: 0.833

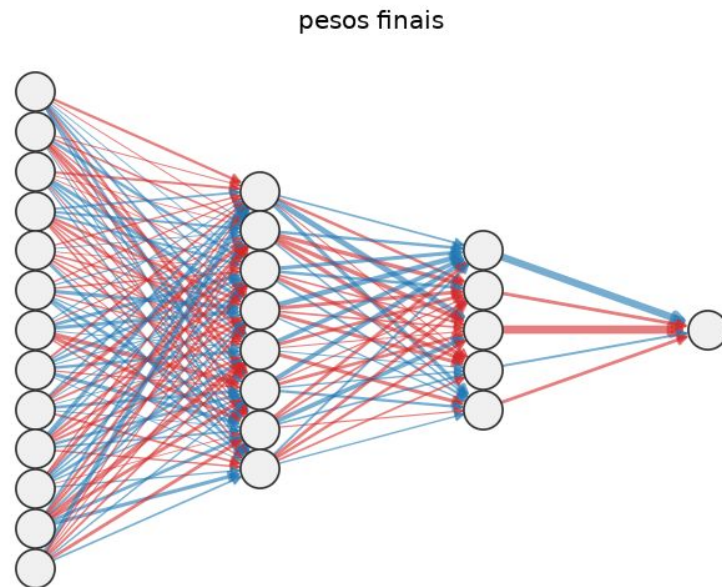
learning rate: 0.005

activation: relu

architecture: 13 | 8 | 5 | 1

pre-processing: standardize

comentários: MLP codificada do zero como grafo (networkx), com feed forward e back propagation manuais — correteude do backward provada por gradient checking. Redes maiores/mais épocas chegavam a 100% no treino mas caíam no teste; a arquitetura enxuta (8|5, 60 épocas) foi o que generalizou melhor.



0.604 : Andreza Maria Coutinho Falcão

accuracy: 0.604

learning rate: 0.05

activation: relu (ocultas) + softmax (saída)

architecture: 166 | 32 | 16 | 3

pre-processing: min-max + one-hot de categóricas
(parâmetros ajustados só no treino) + códigos ICD-9
agrupados em grandes grupos diagnósticos

comentários: MLP codificada do zero em NumPy puro,
com feed forward e backpropagation manuais — regra da
cadeia camada a camada; softmax+cross-entropy
simplifica o gradiente da saída para $(\hat{y}-y)/N$.